Unisync — UX & Tech Stories

Version 5.0 · FINAL — Signed off 10 August 2026 Status: FROZEN. This document is the signed source of truth for Unisync app UX and supporting firmware behavior. Changes after this point require a new version (5.1+) with the change and its reason recorded — no silent edits.

Covers standalone-master operation (a master not joined to a mesh) unless stated otherwise. All product decisions in this document are made; the Open items section contains only engineering specs to be written and test coverage to be added, each of which is an output of this document, not a blocker to it.

Sign-off

Role	Name	Date
Product Owner		10 Aug 2026
Engineering (firmware)		
Engineering (app)		
QA		

How Unisync works (context for every story below)

There is no home router. The Master broadcasts its own password-protected Wi-Fi network — that network *is* the product. The phone joins it directly; the phone's internet stays on mobile data. The Master also advertises Bluetooth at all times. The app talks to the Master over whichever one the user prefers.

One password does everything. The factory password printed on the in-box card joins the Wi-Fi network *and* logs into the app. There is no separate account. A separate recovery key, on its own card, exists only for regaining access if the password is lost.

Login produces a token. After login, the app holds a token that works over both Wi-Fi and Bluetooth, survives Master reboots and going out of range, and never expires. The only thing that invalidates tokens is a password change — which invalidates *everyone's*, everywhere, at once.

Control is local only. No router means no internet path to the Master. The app controls the home only from within radio range. There is no remote access, and no story should ever assume one.

Mesh connects masters, nothing else. Multiple masters can join into a mesh — one network name, one password, one home. Mesh is purely master-to-master; each master's relationship with its own extensions is identical in and out of a mesh.

Epic 1 — First-Time Setup

UX Story

As a new user, I want to set up my Master by joining its Wi-Fi network with the password from the box — no router, no Bluetooth required — and land on a working dashboard in a few steps.

Flow

1. Power on the Master. It broadcasts its own Wi-Fi network.
2. Join that network from the phone using the factory password on the in-box card. Open the app.
3. The app detects the Master and logs in with the same password.
4. First dashboard entry prompts: **"Change your password?"** — optional, skippable.
5. The app asks: **"Stay on Wi-Fi, or switch to Bluetooth mode?"** — saved as a persistent preference.
6. User lands on the dashboard. Setup complete.

Acceptance criteria

- **First launch with nothing configured lands on a branded welcome screen** — product branding and a single clear entry point into setup — not an empty dashboard or a bare login form.
- Bluetooth is never required to get from unboxing to a working dashboard.
- One password from the card both joins the network and logs in, first attempt. **The card prints the factory network name alongside the password** — the setup instructions reference a network the user can actually find.
- The Wi-Fi/Bluetooth preference is asked exactly once, at setup — never again on launch.
- The password card and the recovery card are visually and verbally distinct in packaging and in-app copy: two different values, two different jobs.
- Keeping the factory password forever is a permitted, accepted path — the change prompt is optional by policy, not an oversight. Physical possession of the card is treated as equivalent to physical access to the home.
- Password change flow: show "Password changed — reconnecting…", expect the network to drop moments later, rejoin with the new password, log in again, land back where the user was. The confirmation copy says whether the change applied to this master or the whole house.

Connection awareness (core UX rule for the whole app)

The user always knows the connection state without doing anything. The app must never let the user discover a problem by tapping a switch and getting an error.

- A persistent indicator shows the current state: connected via Wi-Fi · connected via Bluetooth · reconnecting · out of range.

- When the phone leaves range or the Master drops, the UI transitions on its own: controls disable, switch states display as "last seen," not as live truth. (In mesh mode, brief roaming handoffs are exempt — see the grace window in Epic 7: only persistent loss surfaces to the user.)
- **Connection problems and login problems are different screens.** Going out of range does not log the user out — the token survives. Show "reconnecting…" for reachability problems; show a login prompt only when login has actually been rejected. A password prompt for a range problem is a bug. A "connection lost" message for a rejected login is a bug.
- On reconnection: resume silently with the existing token, refresh the full state, and only then re-enable controls.

Tech Story

- Master always broadcasts its Wi-Fi network with the active password; always advertises Bluetooth. The mode preference lives in the app and only chooses which channel the app uses.
- **The app must bind its traffic to the Master's network.** Phones treat a Wi-Fi network with no internet as suspect: Android silently routes app traffic back to mobile data unless the app explicitly binds to the network; iOS shows "no internet" warnings. Without binding, requests intermittently fail in ways that look like defective hardware. This is the single most likely field-failure mode of the setup flow.
- Connection state must be driven by events plus a heartbeat, not by waiting for requests to fail: the live-update stream disconnecting is itself the signal, and a periodic lightweight ping catches the case where the link looks alive but the Master is gone. Target: the UI reflects a lost connection within about 5 seconds.
- After a password change, the confirmation is sent *before* the network restarts, so the app always receives it. Every token everywhere dies at that moment — password change doubles as "sign out all devices."

Epic 2 — Switches: Discovery, Naming, State

UX Story

As a user, I want my switches to just appear, keep their names, survive power cuts sensibly, and quietly handle hardware that misbehaves.

Discovery & naming

- Extensions appear on the dashboard automatically with default names — no pairing step, ever.
- User can rename the Master, extensions, and individual switches, and reorder switches, from a menu.
- Names and ordering are the same for every member of the household and survive app reinstalls and new phones.
- Default names never collide — two boards never both show "Switch 1."

Offline extensions — fully automatic, no refresh button

- When an extension becomes unreachable, its switches disappear from the dashboard. It stays in the extension list, marked **offline**, with a last-seen time. When it returns, its switches reappear on their own.
- An extension that keeps dropping and returning is shown as **intermittent** in the list — a distinct diagnostic state — rather than its switches blinking in and out of the dashboard. Offline is declared after three missed check-ins; return to the dashboard requires a solid minute of presence.
- "Offline" means the app can't reach it. The physical switch always keeps working by touch.
- Renaming works even while an extension is offline; the change applies when it returns.
- **An offline extension can be removed** from the extension list — in standalone and mesh mode alike — with a confirmation stating its names and settings will be forgotten. If that physical board later reappears on the bus, it's adopted as new, with default names, like any first-time extension.

Factory reset

- Factory reset is performed with a **physical switch on the back of the box** — never from the app. It returns the master to its out-of-box state: factory network name, card password, empty registry (names, ordering, restore policies, states, extension registrations all wiped). Correct and intended for ownership transfer: the new owner's experience is identical to unboxing.
- The previous owner's app discovers this naturally: the stored session fails, and login with the old password fails — the standard screens handle it. The stale entry can be removed from the switcher like any other.

Power-cycle behavior *(standalone master; mesh behavior specified separately)*

- Each switch has a setting: **restore last state** after a power cut, or **always start off**. It lives in the same menu as rename.
- **Default: always start off.** Nothing energizes after an outage unless the user opted that switch into restore. Because the house comes back dark by default, the restore setting must be introduced during setup or first-run — not left for users to discover after their first outage.
- Restore direction is always safe: a switch may go from off to on as its state is restored, never the reverse — nothing turns on that wasn't on before the outage, and nothing set to "start off" ever restores.
- If the Master fails to start after an outage, everything stays off but every switch still works by touch. A dead Master never strands a dark, untouchable house.
- An extension used standalone (never registered to a Master) behaves as a conventional switchboard: it remembers and restores its own last state through an outage. Restore settings are a smart-mode concept and don't apply here.

Tech Story

- **The Master's persistent storage is the single source of truth** for names, ordering, per-switch restore policy, and last commanded state of every channel. The app is a cache; it never authors truth locally. Factory reset wipes all of it — correct for ownership transfer.
- Rename, reorder, and restore-policy changes are authenticated API mutations, and all of it travels in the live state stream so every connected phone updates together.
- Extension presence (online / offline / intermittent, with last-seen) is determined by the Master from its bus polling and delivered to apps in the state stream. Apps never infer extension presence from their own request failures — an app timeout means the *app* lost the Master, not that an extension left the bus.
- **Registered extensions boot with all relays off** and apply state only when the Master pushes it. On boot the Master reads its registry, waits for extensions to authenticate, then pushes state for restore-enabled channels only — channels set to "start off" need no push at all.
- Restore pushes are staggered a few tens of milliseconds apart per channel to bound inrush current when a whole house comes back at once.
- **Unregistered extensions keep their own last state** in local storage and self-restore on power-up. The stored pairing credential doubles as the registration flag: present → boot off and wait for the Master; absent → self-restore. State tracking runs in both modes, so registering or unregistering mid-life transitions cleanly.
- Relay state writes to persistent storage are debounced (write shortly after the last toggle in a burst, not on every toggle) to limit flash wear.
- Mesh membership changes none of this: mesh is master-to-master only, so each master restores its own board independently, without waiting for the mesh to form.

Epic 3 — Firmware Updates

UX Story

As a user, I want to see firmware versions, check for updates, and download them anytime — but installs happen only while I'm on Wi-Fi. In Bluetooth mode, a downloaded update waits in the app.

Acceptance criteria

- Version display, update check, and download work in both modes. Downloads use the phone's mobile data — the Master has no internet in either mode.
- Installing to the Master or extensions happens only in Wi-Fi mode.
- In Bluetooth mode, a downloaded update shows a clear **"waiting for Wi-Fi"** state.
- Update success is confirmed from the device after it restarts — the app never reports success from the upload alone, because a Master can accept an upload and silently revert on restart.
- An update never logs anyone out; the app never asks for a password around an update.

Tech Story

- The signed-update pipeline is unchanged; this is delivery gating in the app, not a trust-model change. Every image is signature-verified on device regardless of how it arrived.
- After a Master update, the app polls the device's info endpoint post-restart and compares versions before showing success.
- Tokens survive restarts by design, so no re-authentication is needed or should be prompted around updates.

In a mesh

- **One push updates the whole mesh.** A master image pushed to any member propagates master-to-master: each master detects a peer running a newer version and downloads the image from it, applying at its own pace. The user pushes once; the mesh finishes the job.
- **Extension images propagate the same way**: every master receives and stores the pushed extension image, then applies it to its own extensions — matched by extension type — at its own pace.
- The app presents the rollout honestly: the Mesh details list shows each master's current version, and a mesh mid-rollout displays as **updating** progress, not as an error or a version-mismatch warning. Version mismatch is only flagged to the user if a master stays behind after the rollout should have completed.
- Success confirmation follows the same rule per master: version verified from the device after its restart, never assumed from the push.

Epic 4 — Sharing Control

UX Story

As a user, I want to share my password so others in the household can control everything, at the same time as me.

Acceptance criteria

- Sharing the password is the entire model: it joins the network and logs in. No accounts, no invitations, no second secret.
- Any number of people can control the home concurrently; every phone sees every change live.
- The app is honest about the trust model: sharing the password is sharing *full* control — there are no guest tiers and no per-person identity. The only way to un-share is to change the password, which signs out everyone, including the owner. Present that action as what it is — "Reset access" — rather than pretending individual removal exists.

Tech Story

- Concurrency is free: the Master stores no sessions; any number of valid tokens verify independently. There is no user cap and no eviction.
- One token works on every master in a mesh and over both transports — moving through the house never triggers a login.
- Simultaneous control from multiple phones needs a test pass (state stream already broadcasts full snapshots to all clients) but no new engineering.

Epic 5 — Bluetooth Mode

UX Story

As a logged-in user, I want Bluetooth mode to work exactly like Wi-Fi mode for controlling my home — same session, switching modes never asks me to log in again.

Rule: login happens only over Wi-Fi. Bluetooth mode works only while the app already holds a valid session from a Wi-Fi login. There is no login screen in Bluetooth mode.

Acceptance criteria

- A session started over Wi-Fi works immediately over Bluetooth; switching modes never triggers a login prompt.
- If the session becomes invalid while in Bluetooth mode (the password was changed), the app shows a clear path back: **"Your access was reset. Connect to [network name] to log in again"** — an instruction screen, not a dead login form. Bluetooth mode is unusable until the Wi-Fi login completes.
- A user who has never logged in cannot enter Bluetooth mode; the mode toggle explains why rather than failing silently.
- **Permissions are checked before the mode changes, never after.** Switching to Bluetooth mode first verifies the nearby-devices/Bluetooth permission; if missing, the app prompts for it right there, and the mode switches only once granted. A denied permission leaves the user in their current mode with a clear explanation and a path to settings — never a half-switched state or a silent failure.

Tech Story

The token already works over both transports, and Wi-Fi-only login means no password ever crosses Bluetooth — password sniffing and over-the-air brute-forcing are impossible by design, with nothing to build. The device must enforce this, not just the app: the Bluetooth control service accepts token-bearing commands only and has no login operation at all.

The remaining gap — token exposure, and the decided fix. Commands over Bluetooth currently carry the session token in the clear, and the Bluetooth link is open to anyone in radio range. One recorded command would yield a token that works on every master, over both transports, indefinitely.

Decided: per-command proof. On each Bluetooth connection the master issues a random session nonce. Every command carries `HMAC(token, nonce + counter)` instead of the raw token; the master verifies it with the same derivation it already uses for token validation. The token never crosses the air, replays fail on the counter, no pairing dialogs, no per-phone storage on the device — the unlimited-users sharing model is untouched. This reuses the firmware's existing HMAC machinery; like the recovery wrapping, it is protocol assembly, not new cryptography. **Bluetooth mode does not launch before this ships.**

Acceptance criteria

- The Bluetooth service rejects any login-like operation; no code path accepts a password over Bluetooth.
- A session invalidated by password change is rejected over Bluetooth immediately, and the app routes to the Wi-Fi re-login instruction screen.
- No Bluetooth command carries the raw token; a recorded Bluetooth command transcript yields no working token, and a replayed command is rejected.
- The existing Bluetooth recovery flow is untouched by this rule — it is the one deliberate exception to "Bluetooth requires a session," because it exists precisely for users who can't log in. Recovery is fully specified in Epic 8.

Epic 6 — Multiple Masters in One App

UX Story

As a user with more than one Master in my app, I want to switch between them without logging in each time, always land on a truthful screen, and reach the switcher even when disconnected or logged out.

Switching — never blindly show a dashboard. In Wi-Fi mode, selecting Master B while the phone is on Master A's network:

1. The app knows it's on the wrong network and says so plainly: a **connection screen** naming B's network with a prompt to join it — an actionable instruction, not a spinner.
2. Once on B's network (or if B was already reachable), the app verifies the stored session. Invalid → **B's login screen.**
3. Both checks pass → B's dashboard, freshly populated.

A genuinely unreachable Master (network not found, or no answer after joining) gets the standard disconnected screen — distinct copy from the wrong-network prompt, because "connect to B's network" is an instruction the user can follow and "B is out of range" is not.

Acceptance criteria

- With multiple Masters set up, the app opens on the **last-used** Master — subject to the same checks; last-used but unreachable lands on its disconnected screen with the switcher available.
- The master switcher and the Wi-Fi/Bluetooth toggle appear on the login screen and the disconnected screen whenever more than one Master is set up. Being locked out of

one master must never trap the user away from the others.

- Masters that are meshed together are one home: one dashboard, one entry in the switcher — never two switchable entries. (Mesh entries, and how they mix with standalone masters in the switcher, are specified in Epic 7.)
- A failed switch never changes the last-used default; only a successful dashboard entry does.

Tech Story

- **Per-master vault, keyed on the factory UID.** Each entry: UID (permanent identity), display name, network name (a cached hint for instruction copy, reported by the master itself), stored token, preferred mode, last used. Stored in the platform's secure storage; tokens never expire, so a stored token is long-lived whole-home access and is protected accordingly.
- In Bluetooth mode there is no network step; the UID is present in the Bluetooth advertisement, so identity is checked at scan time. Since login is Wi-Fi-only, a Bluetooth-mode switch to a master with no valid stored session routes to the Wi-Fi re-login instruction screen — never to a login form over Bluetooth.
- **Identity is proven by UID via probe — the app never reads the phone's network name.** The gate on switching to master B: probe the device info endpoint. Three outcomes: (1) answers with B's UID → proceed to the session check; (2) answers with a *different* UID → the phone is on another master's network — show the wrong-network prompt: "You're connected to [that master]. Connect to [B's network] to continue"; (3) no answer → disconnected screen. Rejected session → login screen. Only a real rejection ever shows a login prompt. This design needs no OS network-name access and therefore no location permission — the probe is the identity check.
- Network names used in instruction copy come from the masters themselves: each master reports its own current network name via its info endpoint, and the app caches it per UID whenever connected — self-healing after renames, never read from the phone's OS.
- **Wi-Fi switching crosses networks, and the phone OS won't do that silently.** Both platforms interpose prompts or delays on app-initiated network joins. The flow must show an explicit "connecting to [name]…" state, and when the target is in Bluetooth range, offer Bluetooth as the faster path rather than fighting the OS.
- Password-change scope maps to vault cleanup: a device-scope change drops that master's token; a mesh-scope change drops the whole home's — the app removes exactly the right tokens instead of discovering dead ones later.

Epic 7 — Mesh Mode

Mesh is master-to-master connectivity only. Everything between a master and its extensions — discovery, naming, presence, power-cycle restore — is completely unaffected by mesh membership.

UX Story — Creating a Mesh

As a user, I want to turn my standalone master into a mesh by giving it a mesh name and password, and be guided through the reconnection that follows.

Flow

1. User starts mesh creation from either mode (Wi-Fi or Bluetooth) and enters a **mesh name** and **mesh password**.
2. The master restarts its network: the network name becomes the mesh name, the password becomes the mesh password. Every existing session dies at this moment — this is expected, not an error.
3. The app shows a guided transition — "Mesh created. Connect to [mesh name] to continue" — and walks the user to join the new network and log in with the mesh password. This is one continuous flow with clear progress, never a surprise logout or a dead disconnected screen.
4. After re-login, the user lands on the mesh dashboard and can stay on Wi-Fi or switch to Bluetooth mode as usual.

Acceptance criteria

- Creation can be *initiated* from either mode, but the completion path always goes through joining the new network and logging in over Wi-Fi (login is Wi-Fi-only). Starting from Bluetooth mode must lead into the same guided transition, not fail or strand the user.
- The disconnect after creation is pre-announced in the flow ("your network will restart") so the user is never surprised.
- The standalone entry for this master disappears from the app's switcher, replaced by the mesh.

UX Story — Adding a Master to the Mesh (2-click join)

As a user, I want to add a new master to my mesh with two actions: one tap to start, one tap to confirm — the app and the masters handle everything else.

Flow

1. **Click 1 — "Add master".** The app silently fetches a joining PIN and the current master's identity from the mesh. These are never shown to the user — they stay encapsulated in the app.
2. The app prompts: **"Connect your phone to the network of the master you want to add"** — with a **Confirm** button. For a factory-fresh master, that's the network name and password from its box; for a master that's been used standalone, it's whatever network name and password the user gave it. The copy accommodates both; no reset is ever required to add a used master, and its names, extensions, and settings all survive the join.
3. **Click 2 — Confirm**, after the user has joined the new master's network. The app sends the join request — carrying the encapsulated PIN and identity — to the new master,

which registers itself as a peer on the mesh.

4. The new master restarts into the mesh network. The app guides the phone back onto the mesh network (credentials already known — this return should be automatic where the platform allows) and lands on the mesh dashboard, now showing the new master's card.

Acceptance criteria

- **Hard cap: two in-app taps, total.** Tap 1 — "Add master." Tap 2 — Confirm after joining the new master's network. No other tap, acknowledgment, "next" button, or dismissal exists anywhere in the flow; every other screen is informational or auto-advancing. Operating-system network dialogs are outside this count — they're acceptable and expected — but they never justify adding an in-app tap around them.
- Everything else — PIN handling, join request, network transitions — is automatic or guided with clear progress states. No raw PINs, identities, or technical values are ever shown.
- Joining a mesh requires Wi-Fi mode. Bluetooth mode must not offer or allow mesh joining.
- No UI flapping: the flow is a single continuous guided sequence with progress indication, not a series of disconnected screens the user stumbles between.
- A failed join (wrong network, PIN rejected, new master unreachable) lands on a specific, actionable error state that offers retry — never a generic failure or a silent return to the dashboard. (Retry taps on the error path are outside the two-tap cap, which applies to the successful flow.)

UX Story — Living in a Mesh

As a mesh user, I want every master's switches on one dashboard, full control from either mode, and the same connection-awareness standards as standalone mode.

Acceptance criteria

- **Dashboard: one collapsible card per master, one card expanded at a time.** Expanding a card collapses the previous one.
- **Bluetooth mode controls the whole mesh.** Once logged in, switching to Bluetooth mode gives control of every master's switches — not just the nearest one — with no added lag and no stale states relative to Wi-Fi mode.
- **Mesh rename and mesh password change are Wi-Fi-only** operations. Both restart the network; both reuse the standard guided reconnect flow from Epic 1 (pre-announced drop, rejoin, re-login where required) — identical handling to standalone mode.
- **Switch reordering inside a master's card** persists on that master, same as standalone mode — every household member sees the same order.
- **Master-card reordering is a personal, app-local preference.** Different household members want different masters on top (usually their nearest one), so card order

deliberately does not sync between users or survive app reinstall. This is the one intentional exception to "the master owns all ordering truth."

- **A master going offline or online in the mesh is handled without UI flapping**: its card shows an offline state (switches disabled, last-seen time) rather than vanishing, transitions are debounced the same way extension presence is (declared offline after missed check-ins, restored after solid presence, "intermittent" if cycling), and the user always sees it before acting — never discovers it through a failed tap.

Roaming — moving through the house

- **Roaming is invisible.** As the user walks through the home, control keeps working in both modes with no reconnect prompts, no login prompts, and no user action. The app hands off between masters on its own.
- **Wi-Fi mode:** all mesh masters broadcast the same network, so the phone's own Wi-Fi roaming does the physical handoff — the app's job is to survive it silently. Handoffs must not flash a "reconnecting" state: brief transitions resolve invisibly, and switch states never go stale or blank during one. On entering mesh mode, the app shows a one-time tip: **keep Wi-Fi auto-connect/auto-join enabled for [mesh name]** for seamless roaming — a suggestion the user sees once, not a nag.
- **Bluetooth mode: zero user action, full stop.** The app continuously tracks which mesh masters are in range and moves its connection to the best one as the user moves. The user never picks a master, never confirms a handoff, never notices one happened.
- Control continuity target: a command issued mid-handoff, in either mode, still lands — at worst with a moment's delay, never with an error the user has to retry.

UX Story — Managing the Mesh

As a mesh user, I want to see every master in my mesh and remove one when it's leaving the home.

Acceptance criteria

- A **Mesh details** menu lists all member masters with name, online/offline state, and firmware version, and offers per-master actions — including **Remove from mesh**.
- Removing a master (or a master leaving) is explicit and confirmed — copy states plainly what happens: that master restarts as a standalone device with the network name and password it had before joining, and its switches leave the mesh dashboard.
- The removed master keeps everything of its own: its extensions, names, ordering, restore policies, and state are untouched — leaving a mesh is a network divorce, not a reset.
- After removal, it reappears in the app's switcher as a standalone master (its old vault entry, restored); the user logs into it with its standalone password.
- Removing the last other master leaves a one-master mesh, which is valid; dissolving the mesh entirely (the last master reverting to standalone) is the same action applied to the final member.

UX Story — Multiple Meshes and the Switcher

As a user, I want meshes and standalone masters in one switcher, clearly told apart, with the same seamless switching.

Acceptance criteria

- The app switcher lists standalone masters and meshes together, each entry visibly typed (standalone vs. mesh) — never ambiguous.
- **Masters that belong to a mesh never appear as individual switcher entries** — the mesh is the entry.
- Switching to a mesh follows the exact standalone gates: wrong-network → connection screen naming the mesh network with a join prompt; unreachable → disconnected screen; invalid session → login screen (or, in Bluetooth mode, the Wi-Fi re-login instruction screen); all checks pass → mesh dashboard.
- Last-used behavior is uniform: the app opens on the last-used entry, mesh or standalone, subject to the same checks.
- All connection-state handling matches the app-wide rule: the app knows and shows before the user acts — no "oops" discovered through a failed tap.

Tech Story

- **Mesh creation is a password-change-plus-rename in one operation**: network name ← mesh name, password ← mesh password, network restarts, every token everywhere dies. Reuse the existing password-change mechanics (confirmation sent before the restart) so the app can drive the guided transition rather than inferring it from a dropped connection.
- **Vault changes on creation:** the standalone entry for that master is retired and replaced by a mesh entry. The mesh entry needs a **stable mesh identity** to key on — the mesh name is user-changeable, so it cannot be the key (same lesson as network names in Epic 6). A durable mesh identifier that survives rename, exposed via the device info endpoint, is a small firmware spec this feature needs.
- **Join protocol:** the join PIN and master identity are fetched over the authenticated session, held in app memory only (never rendered, never logged), and submitted to the new master over its factory network. The new master validates the PIN through the existing mesh enrolment protocol — including its failed-attempt lockout — and registers as a peer. After joining, its own registry survives intact: its extensions, names, restore policies, and state all remain its own, because mesh never touches master-to-extension behavior.
- **Bluetooth whole-mesh control:** in Bluetooth mode the app connects to whichever mesh master is in range; that master relays commands to peers over the mesh, and the state stream delivered over Bluetooth carries the *whole mesh's* state, not just the connected master's. The no-lag/no-staleness requirement makes this relay path a first-class citizen: cross-mesh command latency and state propagation over the Bluetooth entry point need targets and tests, not just "it works."

- **Master presence in the mesh** reuses the extension-presence pattern at the mesh level: the mesh tracks peer liveness, presence states (online / offline / intermittent, last-seen) travel in the state stream, and debounce thresholds prevent card flapping. Apps never infer a *peer's* presence from their own connection health.
- **Roaming mechanics.** The token works on every mesh master by design, so handoffs never re-authenticate — roaming is purely a transport concern. Wi-Fi mode: when the phone re-associates to a different master, the app detects the state-stream drop, immediately reconnects (any master answers — same network, same token), pulls a fresh full state snapshot, and reconciles silently. Bluetooth mode: the app scans in the background, ranks in-range mesh masters by signal, and migrates its connection when a meaningfully better one appears — with hysteresis, so it never ping-pongs between two masters of similar strength. Commands issued during a migration are queued briefly and sent on the new connection rather than failed.
- **Grace window for transition states** *(refines the Epic 1 connection-awareness rule)*: the ~5-second "show the user" target applies to *persistent* loss. During roaming handoffs, the app holds the last-known state and connection indicator steady for a short grace period (a few seconds); only a drop that outlives the grace window surfaces as "reconnecting." This keeps the no-flap guarantee and the always-aware guarantee from fighting each other: silent when the handoff succeeds, honest when it doesn't.
- The auto-connect tip is platform-specific under the hood (auto-join on one platform, saved-network behavior on the other) but presented as one plain sentence; shown once on first mesh entry, dismissible, findable again in settings.
- **Switch order lives on the owning master** (as standalone); **master-card order lives in app storage only**, per user, by design.
- **Leaving or being removed from a mesh is credential revocation, enforced cryptographically.** The departing master reboots with its retained standalone credentials (kept in its storage from before the join, exactly as the pre-mesh state). The remaining mesh rotates or invalidates the departed member's mesh credential so the departed master — or anyone who extracted its stored secrets — can no longer participate in or listen to mesh traffic. "Kicked means deaf": exclusion is enforced by the mesh, not by trusting the departed device to behave.
- Mesh rename and password change follow the same restart-with-prior-confirmation mechanics as creation; scope for token invalidation is the whole mesh.

Epic 8 — Recovery

Recovery is for the user who has lost their password. It works over Bluetooth — the one place Bluetooth accepts an unauthenticated flow, because recovery exists precisely for people who cannot log in. It recovers whatever the target master belongs to: connect to a meshed master and the *mesh* is recovered; connect to a standalone master and that *device* is recovered.

UX Story

As a locked-out user, I want to recover my home with the recovery key from the card and set a new password in one step, from the login screen, without needing any working credentials.

Flow

1. From the login screen (the user is locked out by definition), the user opens **Recover access** and the app connects to the master over Bluetooth.
2. The recovery screen asks for two things: the **recovery key** (from the recovery card) and a **new password**.
3. User taps **Recover**. The master answers plainly: accepted or rejected.
4. **Rejected:** the app says so, and each successive rejection doubles the wait before another attempt is allowed, starting at 2 seconds. The wait is visible (a countdown on the button), so the slowdown reads as security, not breakage.
5. **Accepted:** the app confirms — "Done. Your home is being recovered." — and everything after that is the master's job: recovering the mesh across its peers, or resetting the standalone device. The app then guides the user into the standard flow: connect to the network with the new password and log in.

Acceptance criteria

- Recovery is reachable from the login screen and the disconnected screen — never buried behind anything that itself requires being logged in.
- Scope is automatic and communicated: recovering a meshed master says the *whole home* is being recovered; a standalone master says *this device*. The user never chooses a scope.
- The accepted/rejected outcome is immediate and unambiguous — no silent timeouts, no spinner that never resolves.
- The backoff after rejections is enforced by the device and visible in the app: each rejection tells the app exactly how long to wait, and the app shows that countdown on the button. Trying another mesh master during a lockout continues the same countdown — the whole home is one recovery gate.
- After acceptance, the app does not pretend to know recovery progress on other mesh masters — it says recovery is underway and moves the user to reconnecting with the new password. If the network restart takes a moment, the standard guided reconnect handles it.
- The screen asks for **the recovery key of the specific master the phone is connected to** — the copy names that master ("the recovery card for [master name]") so a household with several cards grabs the right one. The key of any *other* master, meshed or not, produces the rejected state. Recovering a meshed master with its own key still recovers the whole mesh; the key just has to belong to the master answering the Bluetooth connection.

- The recovery card and password card stay clearly distinct in the copy — the screen asks for the *recovery key*, and mistaking one card for the other produces the rejected state, not confusion.

Tech Story

- **The new password never crosses Bluetooth in the clear.** Hard requirement, not an option: the Bluetooth link is open, and this flow transmits the next whole-home password. The recovery key is a secret shared between the card and the device, so the request is authenticated by proof-of-knowledge of the recovery key (challenge–response, as the existing recovery service already does) and the new password travels wrapped — encrypted with a key derived from the recovery key. A recorded recovery exchange must yield neither the recovery key nor the new password. The firmware's existing wrapping primitives cover this; it is protocol assembly, not new cryptography.
- **Backoff is enforced by the master and carried in its response.** The 2-second-doubling schedule lives in firmware, and every rejection response includes the wait remaining before the next attempt is allowed — the app just renders the countdown it was given and never tracks or computes backoff itself. The attempt counter is synchronized across the mesh (or held by the standalone device), so connecting to a different mesh master mid-lockout continues the same backoff rather than resetting it — the mesh presents one recovery gate, not one per master. The master refuses early retries regardless of what a client sends; app behavior is presentation only.
- Explicit accepted/rejected replaces the older silent-timeout behavior — a product decision. The recovery key's entropy plus device-enforced exponential backoff is what makes an explicit answer safe to give.
- **Scope resolution is the master's decision, not the app's:** the target master knows whether it is meshed. Accepted-on-meshed triggers mesh-wide recovery — the master propagates the new credential to its peers and the mesh network restarts under the new password; the network *name* is unchanged, so the reconnect targets the same network with new credentials. Accepted-on-standalone resets that device and restarts its network.
- Recovery is a password change, with everything that implies: every token in its scope dies, and the app's vault cleanup follows the same scope mapping as any password change.
- The recovery service remains reachable regardless of login state, and remains the sole unauthenticated Bluetooth flow — the deliberate exception already stated in Epic 5.

Open items

All product decisions in this document are made. What follows is implementation work this document creates: engineering specs to write and test coverage to add.

Launch gate: Bluetooth mode does not launch until the per-command proof (Epic 5, decided) is implemented and its acceptance tests pass.

High risk, unbuilt

1. **Network binding to a no-internet Wi-Fi network (Epic 1)** — unbuilt and untested on both platforms; the most likely source of "flaky hardware" complaints. The same platform investigation covers multi-master network switching (Epic 6) — do it once.

Small specs needed (uncontentious) 2. Heartbeat interval and timeout for connection liveness (Epic 1). 3. Presence fields (online/offline/intermittent, last-seen) in the state stream and extensions endpoint (Epic 2). 4. Naming/ordering/restore-policy endpoints and storage layout; extension-remove endpoint (Epic 2). 5. Restore stagger interval and state-write debounce interval (Epic 2). 6. A stable mesh identifier that survives mesh rename, exposed via the device info endpoint, for the app's switcher and vault; the info endpoint also reports the master's own current network name for instruction copy (Epics 6, 7). 7. Latency and state-propagation targets for whole-mesh control through a single Bluetooth-connected master (Epic 7). 8. Roaming parameters: handoff grace-window duration, Bluetooth signal-ranking hysteresis thresholds, and mid-handoff command queue timeout (Epic 7). 9. Mesh member removal: credential rotation/invalidation on kick or leave, so a departed master cannot rejoin or listen ("kicked means deaf") (Epic 7). 10. Recovery protocol assembly: challenge–response plus new-password wrapping under a recovery-key-derived key, and the device-side backoff schedule with mesh-synced counter (Epic 8). 11. Bluetooth per-command proof protocol: session nonce issuance, counter handling, and device-side verification (Epic 5).

New test coverage required 12. Power-cycle state restore — registered and standalone extension paths (nothing covers this today). 13. Unregistered-extension boot branch (existing bring-up tests assume the registered path). 14. Concurrent multi-phone control (Epic 4). 15. The Epic 5 acceptance list, once built, including permission-denied paths on both platforms. 16. Wrong-network / probe-identity switching flows, including stale vault entries after a factory reset (Epic 6). 17. Mesh creation and 2-click join end-to-end — factory-fresh *and* previously-used masters — including guided network transitions and failed-join error states (Epic 7). 18. Mesh member removal: departed master reverts to its standalone credentials with registry intact, and is verifiably excluded from mesh traffic afterward (Epic 7). 19. Whole-mesh control and state freshness over Bluetooth; mesh peer offline/online card behavior under flapping conditions (Epic 7). 20. Roaming walk-test in both modes: continuous control while moving through all masters' coverage, commands issued mid-handoff, no visible reconnect states on successful handoffs (Epic 7). 21. Mesh firmware rollout: one push propagates to all masters; extension images stored by every master and applied by type; per-master version verification post-restart (Epic 3). 22. Recovery: recorded exchange yields no recovery key and no new password; device-enforced backoff holds against a client that ignores it; backoff continues, not resets, when retrying against a different mesh master; the connected master's own key recovers, any other master's key rejects; mesh-wide recovery propagates to all peers; standalone recovery resets the single device (Epic 8). 23. Extension removal and re-adoption: removed extension disappears; the same physical board reappearing is adopted fresh with defaults (Epic 2). 24. Factory reset via the physical switch: full registry wipe, factory credentials restored, prior app sessions and logins fail cleanly (Epic 2).